Cybersecurity Lab Report: TCP SYN Flood DoS Attack Analysis

1. Executive Summary

This report documents a hands-on cybersecurity laboratory simulating a **TCP SYN Flood Denial of Service (DoS)** attack. The experiment evaluates how a target operating system (Windows) handles intensive Layer 4 protocol stress across different port states and application-layer configurations. All sensitive infrastructure information has been fully anonymized for safe, public-facing deployment on GitHub portfolios.

2. Lab Environment Setup

- **Attacker Machine:** Kali Linux (Aisolated Host-Only Interface)
- **Victim Machine:** Windows OS (Target IP: `[TARGET_IP]`)
- **Tooling:** `hping3` (Network packet crafting), `netstat` (Network diagnostics)

3. Experimental Performance Metrics

The following table encapsulates the real-time resource exhaustion metrics collected via the target's Performance Monitor across three distinctive testing baselines:

Test Scenario	CPU Usage	RAM Usage	Network Load	Disk I/O	Target Behavior / Status
Port 0 Inundation	**1% → 20%** (in 2 seconds)	Baseline	3%	1%	Packets silently dropped at the Kernel level. No socket allocation.
Port 80 (Closed/No Web Server)	**24%** (Constant)	32%	3%	1%	OS continuously issues TCP RST packets. Network stack overhead observed.
Port 80 (Active Web Page & Service)	**44% → 90%** (Critical Spike)	36% → 40%	3%	1% → 5%	Application-layer thread exhaustion. HTTP service went unresponsive.

4. Technical Breakdown & Analysis

- **Port 0 Mechanics:** Targeting Port 0 triggered a rapid CPU spike up to 20% within 2 seconds. Because Port 0 is an invalid transmission port, the operating system's network driver drops these incoming packets almost immediately, preserving Memory and Disk integrity.
- **Port 80 without Application Layer:** Sending SYN requests to a closed Port 80 forced the CPU to stick at 24% and RAM at 32%. The OS network stack had to actively process each packet to reject it, consuming processing power despite no web server running.
- **Port 80 with Active Service:** When attacking an open port hosting an active landing page, CPU usage severely escalated from 44% up to 90%, while RAM crept up to 40%. The web server's connection pool became

exhausted, and Disk I/O spiked to 5% due to the concurrent web access tracking and error logging engines.

5. Implementation Commands

To run the application-layer attack scenario without target IP leakage, the following syntax was deployed from the attacker node:

```
sudo hping3 -S --flood -p 80 [TARGET_IP]
```

To audit connection backlog table saturation directly on the victim machine:

```
netstat -an -p tcp
```

Observation: The connection state output flooded with `SYN_RECEIVED` status loops pointing explicitly back to the single, non-spoofed attacker interface.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.